

DOCUMENTACIÓN DEL MODELO DE BASE DE DATOS

Sistema de Venta y Mantenimiento de Vehículos

Este documento describe el diseño del modelo de datos para un sistema de gestión de ventas y mantenimiento de vehículos. Se presentan dos representaciones: el diagrama entidad-relación (notación de Chen) y el modelo relacional implementable en una base de datos. Se justifican las decisiones de diseño, las restricciones de integridad y las relaciones entre las entidades.

1. Justificación del Diseño

1.1 Estructura general de las tablas

El modelo se organiza en seis tablas principales que representan las entidades del negocio: **CLIENTE**, **VENDEDOR**, **VENTA**, **VEHICULO**, **MANTENIMIENTO** y **DETALLE_VENTA**. Esta separación permite gestionar de forma independiente los actores (clientes y vendedores), los productos (vehículos), las transacciones (ventas) y los servicios posteriores (mantenimientos).

1.2 Decisiones de diseño clave

- **Clave primaria de VEHICULO:** Se utiliza **id_vehiculo** (entero autoincremental) como clave primaria en lugar del VIN. El VIN se mantiene como atributo UNIQUE porque, aunque es un identificador natural único, es una cadena larga. Usar un entero simplifica las claves foráneas, mejora el rendimiento de los índices y facilita la integridad referencial.
- **Tabla DETALLE_VENTA (relación N:M):** La relación entre VENTA y VEHICULO es de muchos a muchos. Por ello se creó la tabla de unión **DETALLE_VENTA** con clave primaria compuesta (**id_venta** + **id_vehiculo**) y el atributo adicional **precio_venta**, que permite registrar el precio pactado en esa transacción específica.
- **Mantenimiento opcional para el cliente:** En **MANTENIMIENTO** el campo **id_cliente** admite NULL. Esto permite registrar servicios a vehículos aunque el cliente no esté identificado (vehículos de demostración o de flota).
- **Separación de responsabilidades:** Los datos del vendedor se mantienen en su propia tabla. La factura y el método de pago pertenecen a **VENTA**. Los datos técnicos del vehículo se concentran en **VEHICULO** para evitar redundancia.

1.3 Mapeo desde el diagrama entidad-relación (Chen)

El diagrama de Chen define las relaciones conceptuales. Cada una se traduce al modelo relacional así:

Relación Chen	Cardinalidad	Implementación Relacional
Realiza (Cliente – Venta)	1 : N	FK id_cliente en VENTA
Registra (Vendedor – Venta)	1 : N	FK id_vendedor en VENTA
Incluye (Venta – Vehículo)	N : M	Tabla DETALLE_VENTA (PK compuesta)
Solicita (Cliente – Mantenimiento)	1 : N	FK id_cliente en MANTENIMIENTO (nullable)
Recibe (Vehículo – Mantenimiento)	1 : N	FK id_vehiculo en MANTENIMIENTO

2. Restricciones y Validaciones

2.1 Claves Primarias (Primary Keys)

Cada tabla posee una clave primaria que garantiza la unicidad de cada registro:

Tabla	Clave Primaria	Tipo	Observaciones
CLIENTE	id_cliente	INT	Autoincremental
VENDEDOR	id_vendedor	INT	Autoincremental
VENTA	id_venta	INT	Autoincremental
VEHICULO	id_vehiculo	INT	Autoincremental (no se usa el VIN)
MANTENIMIENTO	id_mantenimiento	INT	Autoincremental
DETALLE_VENTA	id_venta + id_vehiculo	INT + INT	Clave primaria compuesta

2.2 Claves Foráneas (Foreign Keys)

Las claves foráneas mantienen la integridad referencial entre tablas:

- **VENTA.id_cliente** → CLIENTE.id_cliente (toda venta pertenece a un cliente existente)
- **VENTA.id_vendedor** → VENDEDOR.id_vendedor (toda venta es registrada por un vendedor)
- **DETALLE_VENTA.id_venta** → VENTA.id_venta
- **DETALLE_VENTA.id_vehiculo** → VEHICULO.id_vehiculo
- **MANTENIMIENTO.id_vehiculo** → VEHICULO.id_vehiculo (obligatorio)
- **MANTENIMIENTO.id_cliente** → CLIENTE.id_cliente (opcional / NULL permitido)

2.3 Restricciones de Unicidad y otras validaciones

- **VEHICULO.vin**: UNIQUE. Aunque no es la clave primaria, el número de identificación del vehículo debe ser único. Se declara UNIQUE para impedir duplicados.
- **VENDEDOR.numero_empleado**: UNIQUE. Cada empleado tiene un número interno único.
- **VENTA.num_factura**: Se recomienda tratarlo como UNIQUE para evitar facturas duplicadas.
- **Campos NOT NULL**: Las claves primarias y la mayoría de atributos de negocio no admiten valores nulos. Solo **MANTENIMIENTO.id_cliente** está definido como nullable.
- **Tipos de datos**: Se eligieron tipos precisos (DECIMAL(10,2) para montos, DATETIME para fechas con hora, BOOLEAN para disponibilidad) para garantizar validación a nivel de motor.

3. Relaciones entre Entidades

A continuación se explica en detalle cada relación presente en ambos diagramas (notación de Chen y modelo relacional).

3.1 Relación «Realiza» (Cliente – Venta)

Un cliente puede realizar muchas ventas a lo largo del tiempo (cardinalidad 1:N). Cada venta pertenece a un único cliente. Se implementa colocando la clave foránea **id_cliente** dentro de la tabla VENTA. La integridad referencial asegura que no se pueda registrar una venta de un cliente inexistente.

3.2 Relación «Registra» (Vendedor – Venta)

Cada venta es registrada por un solo vendedor, pero un vendedor puede registrar múltiples ventas (1:N). Se implementa con la FK **id_vendedor** en VENTA. Esto permite conocer el desempeño de cada vendedor y generar reportes de comisiones.

3.3 Relación «Incluye» (Venta – Vehículo)

Esta es una relación de muchos a muchos (N:M). Una misma venta puede incluir varios vehículos y un vehículo podría estar asociado a distintas ventas históricas. Por esta razón se crea la tabla intermedia **DETALLE_VENTA**. Además de las dos claves foráneas que forman la PK compuesta, se almacena **precio_venta**, atributo propio de la relación (el precio real al que se vendió el vehículo en esa transacción).

3.4 Relación «Solicita» (Cliente – Mantenimiento)

Un cliente puede solicitar muchos servicios de mantenimiento (1:N). Se refleja con la FK **id_cliente** en MANTENIMIENTO. Se permite el valor NULL para cubrir casos en los que el servicio se realiza sin un cliente registrado (vehículos de la empresa, demostraciones).

3.5 Relación «Recibe» (Vehículo – Mantenimiento)

Cada mantenimiento se aplica a un vehículo concreto, y un vehículo puede recibir múltiples mantenimientos a lo largo de su vida (1:N). Se implementa con la FK **id_vehiculo** obligatoria en MANTENIMIENTO. Esto permite llevar un historial completo de servicios por vehículo.

3.6 Resumen de cardinalidades

Entidad A	Relación	Entidad B	Cardinalidad
Cliente	Realiza	Venta	1 : N
Vendedor	Registra	Venta	1 : N

Venta	Incluye	Vehículo	N : M
Cliente	Solicita	Mantenimiento	1 : N
Vehículo	Recibe	Mantenimiento	1 : N

4. Información Adicional del Modelo

4.1 Listado completo de atributos por tabla

CLIENTE: id_cliente (PK), nombre, telefono, correo, direccion

VENDEDOR: id_vendedor (PK), nombre, numero_empleado (UNIQUE), fecha_contratacion

VENTA: id_venta (PK), num_factura, fecha_venta, total, metodo_pago, id_cliente (FK), id_vendedor (FK)

VEHICULO: id_vehiculo (PK), marca, modelo, anio, vin (UNIQUE), precio, color, tipo_combustible, tipo_transmision, estado, disponible

MANTENIMIENTO: id_mantenimiento (PK), tipo_servicio, costo, fecha_servicio, id_vehiculo (FK), id_cliente (FK, NULL)

DETALLE_VENTA: id_venta (PK/FK), id_vehiculo (PK/FK), precio_venta

4.2 Beneficios del diseño

- Normalización hasta 3FN: se evita la redundancia de datos de clientes, vendedores y vehículos.
- Integridad referencial garantizada por las claves foráneas.
- Flexibilidad para ventas de uno o varios vehículos gracias a DETALLE_VENTA.
- Historial completo de mantenimientos por vehículo y, opcionalmente, por cliente.
- Facilidad para generar reportes de ventas por vendedor, por periodo, por marca, etc.

4.3 Consideraciones de implementación

Se recomienda definir índices sobre las claves foráneas (id_cliente, id_vendedor, id_vehiculo) para acelerar las consultas de join. El campo vin debe tener un índice UNIQUE. En entornos de producción se pueden añadir triggers o reglas de negocio adicionales (por ejemplo, impedir la venta de un vehículo cuyo campo disponible = FALSE).